



Universidad Distrital Francisco José de Caldas
Systems engineering, school of Engineering

Technical Report: Game Variant Fire Brigade

R. Andres Diaz Lozada, A. Mauricio Cepeda Villanueva, J. Felipe Cuervo
Huertas

Supervisor: Carlos A. Sierra Virgüez

June 4, 2026

Declaration

We, R. Andres Diaz Lozada, A. Mauricio Cepeda Villanueva, J. Felipe Cuervo Huertas of Systems Engineering, Faculty of Engineering, Francisco José de Caldas District University, confirm that this is our own work, and that figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where such works have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism and penalized accordingly. We give consent to a copy of our report being shared with future students as an exemplar. We give consent for our work to be made available more widely to members of Universidad Distrital Francisco José de Caldas and to the public with interest in teaching, learning, and research.

R. Andres Diaz Lozada, A. Mauricio Cepeda Villanueva, J. Felipe Cuervo Huertas
June 4, 2026

Abstract

This report presents the design and development of Fire Brigade, a turn-based simulation game that models forest fire suppression and civilian evacuation on an 8×8 grid. The project implements two fundamental algorithms: a Greedy algorithm for automatic firefighting unit dispatch to the highest-risk cell, and a Backtracking algorithm for finding safe evacuation routes avoiding burning cells. The technical architecture separates the game engine (C++) from the algorithmic logic (Python), communicating via JSON and presenting the interface through Pygame. Key design decisions include the use of a Binary Search Tree (BST) to maintain cell risk levels for $O(\log n)$ access, and a Linked List to manage the unit dispatch queue. The game incorporates three difficulty levels, three unit types (Crew, Truck, Helicopter), four civilian types (Child, Adult, Elderly, Family Group), and two fire behaviors (Normal and Canopy). This report details the system architecture, data structures, algorithms, and the expected outcomes for the complete implementation.

Keywords: Greedy Algorithm, Backtracking, Binary Search Tree (BST), Fire Suppression, Civilian Evacuation, Python, PyGame, C++, Data Structure, Linked Lists.

GitHub repository <https://github.com/MaurooC12/Fire-Brigade>

Contents

List of Figures	v
List of Tables	vi
1 Introduction	1
2 Literature Review	2
2.1 Greedy Algorithms	2
2.2 Backtracking Search	2
2.3 Linked Lists	3
2.4 Binary Search Trees	3
2.5 Summary	3
3 Objectives	4
3.1 General Objective	4
3.2 Specific Objectives	4
4 Scope	5
4.1 Included in this Study	5
4.2 Excluded from this Study	5
5 Assumptions and Limitations	7
5.1 Assumptions	7
5.2 Limitations	8
6 Methodology	9
6.1 System Architecture	9
6.2 Data Structure Design	10
6.2.1 Unit Dispatch Queue — Singly Linked List	10
6.2.2 Risk-Ranking Structure — Binary Search Tree	11
6.3 Algorithm Design	12
6.3.1 Greedy Dispatcher	12
6.3.2 Backtracking Evacuator	12
6.3.3 Complexity Analysis	13
6.4 Game Components	14
6.4.1 Unit Types	14
6.4.2 Civilian Types	14
6.4.3 Fire Behavior	14
6.5 Game Rules	15
6.5.1 Scoring System	15
6.5.2 Win Conditions	15
6.5.3 Loss Conditions	15

6.5.4	Turn Structure	16
6.6	Difficulty Levels	16
6.7	JSON Communication Protocol	17
6.7.1	input.json (Python → C++)	17
6.7.2	state.json (C++ → Python)	17
6.8	User Interface Wireframe	18
7	Implementation & Results	19
7.1	Implementation Status	19
7.2	Planned Implementation	19
7.3	Current Implementation Screenshots	20
7.3.1	Main Menu	20
7.3.2	Game Grid	20
7.3.3	Victory Screen	21
7.3.4	Defeat Screen	21
7.4	Expected Results	21
7.5	Current Results	22
8	Conclusion and Reflections	23
8.1	Conclusion	23
8.2	Reflections	23
8.2.1	Design Decisions	23
8.2.2	Challenges Encountered	24
8.2.3	Lessons Learned	24
8.3	Future Work	25
	Appendices	26
	References	26

List of Figures

6.1	System Architecture: C++ Engine, JSON Bridge, Python Algorithms, and Pygame UI	10
6.2	Linked List Structure for Unit Dispatch Queue	10
6.3	BST Node Structure and Class Methods	11
6.4	Example BST Nodes with Risk Values and Coordinates	12
6.5	User Interface Wireframe: 8×8 Grid and HUD Panel	18
7.1	Main Menu Screen	20
7.2	8×8 Game Grid with Active Fires and Civilians	20
7.3	Victory Screen – All Fires Extinguished and Civilians Evacuated	21
7.4	Defeat Screen – Loss Condition Triggered	21

List of Tables

6.1	Unit Types and Characteristics	14
6.2	Civilian Types and Values	14
6.3	Fire Types and Spread Patterns	14
6.4	Difficulty Level Configurations	17

List of Abbreviations

BST	Binary Search Tree
FIFO	First-In-First-Out
GUI	Graphical User Interface
HUD	Heads-Up Display
JSON	JavaScript Object Notation
$O(n)$	Big O Notation (linear time complexity)
$O(\log n)$	Big O Notation (logarithmic time complexity)
UI	User Interface

Chapter 1

Introduction

This report presents the design and implementation of a turn-based tactical simulation game set on an 8×8 grid, in which the player commands firefighting and civilian evacuation operations. The game was developed as an academic project to demonstrate the practical application of fundamental data structures and algorithms within an interactive software system.

The project integrates three distinct technological layers: a C++ engine responsible for core data management, a Python module housing the decision-making algorithms, and a Pygame-based graphical interface. Communication between the C++ engine and the Python layer is handled through a JSON Bridge, allowing each component to operate independently while sharing game state efficiently.

At the core of the gameplay mechanics lie two mandatory algorithms. The **Greedy algorithm** drives unit deployment by consulting a Binary Search Tree (BST) maintained by the C++ engine, which ranks all burning cells by risk level and automatically dispatches the selected unit to the highest-risk cell. The **Backtracking algorithm** handles civilian evacuation by exhaustively exploring all possible safe paths from a civilian's position to the grid boundary, avoiding cells currently on fire, and returning the shortest valid route found.

The C++ engine manages two key data structures: a **singly linked list** that implements the unit dispatch queue, storing incoming firefighting units (ground crews, trucks, and helicopters) in order of arrival; and a **Binary Search Tree** that organises burning cells by their computed risk score, enabling the Greedy algorithm to retrieve the maximum-risk target in $O(\log n)$ time.

The game features three difficulty levels — Easy, Medium, and Hard — each varying the number and type of fires, the composition of civilians on the map, the initial unit allocation, and the rate at which new units arrive. Three built-in variants add strategic depth: a *scarce resources* variant governing unit types and arrival timing, a *civilian priorities* variant assigning different point values and movement speeds to four civilian categories, and a *dynamic fire behaviour* variant introducing two fire propagation modes with distinct spread patterns and risk weightings.

The development team was composed of three members, each responsible for a distinct layer of the architecture:

- **Jeison Felipe Cuervo Huertas** — Engine Developer: implemented the C++ core, including the linked list dispatch queue and the BST risk-ranking structure, as well as the JSON Bridge connecting the engine to the Python layer.
- **Andres Mauricio Cepeda Villanueva** — Algorithm Developer: implemented the Greedy dispatcher and the Backtracking evacuator in Python, including tie-breaking logic and path optimisation.
- **Raul Andres Diaz Lozada** — UI Developer: built the Pygame interface, rendering the 8×8 grid, fire propagation animations, evacuation sequences, and the score and resource HUD.

Chapter 2

Literature Review

This chapter reviews the theoretical foundations underlying the two core algorithms implemented in the project — the Greedy strategy and Backtracking search — as well as the data structures that support them: the singly linked list and the Binary Search Tree. Understanding the established literature on these topics justifies the design decisions made during development and contextualises the project within the broader field of algorithm design and data structure theory.

2.1 Greedy Algorithms

A greedy algorithm constructs a solution by making the locally optimal choice at each step, with the expectation that a sequence of local optima will converge toward a global optimum [1]. The defining characteristic of greedy strategies is their irrevocability: once a decision is made, it is never reconsidered. This property makes greedy algorithms computationally efficient, typically achieving $O(n \log n)$ or better time complexity, but also means they are not universally optimal — their correctness depends on whether the problem exhibits the *greedy-choice property* and *optimal substructure* [2].

In the context of this project, the **Greedy Dispatcher** algorithm applies the greedy principle to unit deployment: at every dispatch decision, the unit is unconditionally sent to the cell with the highest current risk score as reported by the BST. This approach prioritises the most dangerous active fire at all times, reflecting a locally optimal strategy for damage containment. While this does not guarantee a globally optimal sequence of deployments across the entire game — since fire propagation introduces dynamic changes to the risk landscape — it provides a consistent, fast, and explainable decision rule that is well-suited to a real-time tactical setting.

Tie-breaking is handled by a secondary greedy rule: among cells sharing the maximum risk value, canopy fire cells are preferred due to their higher propagation rate, and among remaining ties, the cell with the smallest row index followed by the smallest column index is selected, ensuring deterministic behaviour.

2.2 Backtracking Search

Backtracking is a general algorithm design paradigm for solving constraint satisfaction and combinatorial search problems [3]. It performs a depth-first exploration of the solution space, incrementally building candidates and abandoning (backtracking from) any partial candidate that cannot lead to a valid or optimal solution. This avoids the combinatorial explosion of exhaustive enumeration by pruning infeasible branches early [4].

The **Backtracking Evacuator** algorithm implemented in this project applies backtracking to find the shortest safe evacuation route for a civilian on the 8×8 grid. Starting from the civilian's current cell, the algorithm performs a depth-first search across the four cardinal directions, marking visited cells to prevent cycles, and recording the path length each time the grid boundary is reached. Upon finding a complete path, it compares the path length against the current best and retains

the shorter one. After fully exploring a branch, it unmarks the visited cell and removes it from the current path, enabling other branches to reuse that cell — the classical backtracking mechanism. If no boundary-reaching path exists (i.e., the civilian is completely surrounded by fire), the algorithm reports failure and the player is notified.

2.3 Linked Lists

A singly linked list is a dynamic data structure in which each element (node) stores a value and a pointer to the next node in the sequence [1]. Unlike arrays, linked lists support $O(1)$ insertion and deletion at the head or at a known position without requiring element shifting, at the cost of $O(n)$ access by index and additional memory for pointers.

Linked lists are a natural fit for implementing queues, where elements are enqueued at the tail and dequeued from the head [5]. In this project, the C++ engine uses a linked list to model the unit dispatch queue. New firefighting units (ground crews, trucks, helicopters) are appended to the tail as they arrive on a turn-based schedule, and the player dequeues from the front when selecting a unit to deploy. This FIFO ordering preserves the arrival sequence of units, and the dynamic nature of the linked list accommodates the variable arrival rates defined by each difficulty level without pre-allocating a fixed buffer.

2.4 Binary Search Trees

A Binary Search Tree (BST) is a rooted binary tree in which every node satisfies the BST property: all keys in the left subtree are smaller than the node's key, and all keys in the right subtree are larger [1]. This ordering enables search, insertion, and deletion in $O(h)$ time, where h is the tree height. For a balanced BST, $h = O(\log n)$; in the worst case (a degenerate tree), $h = O(n)$.

In this project, each BST node stores the coordinates of a burning cell, its computed risk score, and its fire type. Canopy fire cells receive a risk multiplier of $\times 1.5$ relative to normal fire cells, reflecting their more aggressive eight-directional propagation. The BST is updated after every turn as the fire spreads, new cells ignite, and extinguished cells are removed, keeping the risk ranking consistent with the current game state.

Self-balancing variants such as the AVL tree guarantee $O(\log n)$ height at all times through rotation operations [1]. For this project, a standard BST without automatic balancing was chosen. Given the maximum grid size of $8 \times 8 = 64$ cells, the number of nodes in the tree is at most 64, making the practical performance difference between a BST and an AVL tree negligible. The simpler BST implementation reduces code complexity and is sufficient for the scale of this application.

2.5 Summary

The literature establishes greedy algorithms and backtracking search as complementary paradigms: greedy methods are fast and irrevocable, suited to problems where local optimality is globally sufficient or acceptable; backtracking is exhaustive and reversible, suited to constraint-heavy search problems where correctness is required. The linked list and BST provide the dynamic data management needed to support these algorithms efficiently within the game engine.

Chapter 3

Objectives

3.1 General Objective

To design and develop a turn-based tactical simulation game on an 8×8 grid that demonstrates the practical integration of fundamental data structures and algorithms within a multi-layer software architecture. The system combines a C++ engine — managing a singly linked list for unit dispatch and a Binary Search Tree for fire risk ranking — with a Python algorithm module implementing a Greedy dispatcher for unit deployment and a Backtracking planner for civilian evacuation, and a Pygame graphical interface that renders the game state and animates player actions across three difficulty levels.

3.2 Specific Objectives

- Implement a **Greedy algorithm** that automatically dispatches firefighting units to the cell with the highest fire risk, based on data maintained in a Binary Search Tree.
- Implement a **Backtracking algorithm** that finds safe evacuation routes for civilians, avoiding all burning cells on the 8×8 grid.
- Design and implement a **singly linked list** to manage the unit dispatch queue with FIFO (First-In-First-Out) ordering.
- Design and implement a **Binary Search Tree (BST)** to maintain burning cells ordered by risk level, enabling $O(\log n)$ retrieval of the highest-risk cell.
- Establish a **modular architecture** separating the C++ game engine, Python algorithms, and Pygame interface, communicating via JSON.
- Develop **three difficulty levels** (Easy, Medium, Hard) with varying initial conditions for fires, civilians, units, and arrival rates.
- Incorporate **three gameplay variants** to add strategic depth: scarce resources (unit arrival timers), civilian priorities (different point values and movement speeds), and dynamic fire behavior (normal and canopy propagation).
- Build a **Pygame-based graphical interface** that renders the 8×8 grid, fires, civilians, unit queue, score HUD, and evacuation animations.
- Define a **JSON communication protocol** (input.json and state.json) to enable data exchange between the C++ engine and Python algorithms.

Chapter 4

Scope

4.1 Included in this Study

This report covers the full conceptual design of a turn-based tactical fire simulation game, encompassing the following elements:

- The design of the game mechanics, including turn structure, fire propagation rules, scoring system, and win/loss conditions across three difficulty levels (Easy, Medium, and Hard).
- The specification of the data structures used in the C++ engine: a singly linked list implementing the unit dispatch queue and a Binary Search Tree organising burning cells by risk score.
- The algorithmic design of the Greedy unit dispatcher and the Backtracking evacuation planner, including pseudocode, input/output definitions, and step-by-step logic.
- The proposed three-layer software architecture comprising the C++ engine, the Python algorithm module, and the Pygame graphical interface, and the JSON Bridge communication protocol connecting them.
- The three built-in game variants: scarce unit resources, civilian priority differentiation, and dynamic fire behaviour with two propagation modes.
- A critical discussion of design trade-offs, algorithmic limitations, and potential future improvements.

4.2 Excluded from this Study

The following aspects fall outside the scope of this report:

- **Full software implementation:** the game has not been coded or executed. No source code, compiled binaries, or runtime measurements are presented. All content is design and proposal only.
- **Empirical performance testing:** no benchmarks, execution time measurements, or memory profiling results are included, as the system has not been implemented.
- **User testing and playability evaluation:** no usability studies, player feedback sessions, or gameplay balancing tests have been conducted.
- **Network or multiplayer functionality:** the game is designed as a single-player local application. Online play, multiplayer modes, and networked communication are not considered.

- **Mobile or web deployment:** the proposed interface targets desktop environments via Pygame. Adaptation to mobile platforms, browsers, or game engines such as Unity or Godot is outside the scope of this work.
- **Advanced AI opponents:** the system does not include computer-controlled adversaries or AI players. All decisions are made by a human player, assisted by the Greedy and Backtracking algorithms acting as decision-support tools rather than autonomous agents.
- **Persistent storage and user accounts:** save files, leaderboards, player profiles, and session persistence are not part of the proposed design.

Chapter 5

Assumptions and Limitations

5.1 Assumptions

The following assumptions were made during the design of the game and the elaboration of this report:

- **Grid representation:** the game world is assumed to be fully representable as a discrete 8×8 grid, where each cell occupies exactly one unit of space. No partial occupancy, overlapping entities, or sub-cell movement is considered.
- **Fire propagation determinism:** fire is assumed to spread deterministically every 2 turns, without probabilistic variation. A burning cell always ignites all valid neighbours according to its type (4 directions for normal fire, 8 directions for canopy fire), provided those neighbours are not already burning or protected by a Helicopter. Newly ignited cells inherit 80% of the source cell's intensity.
- **Instantaneous extinguishment:** it is assumed that deploying a unit to a burning cell extinguishes it entirely within the same turn, with no partial or multi-turn suppression process. Additionally, Crew and Truck units reduce the risk of neighboring cells by half upon deployment.
- **Helicopter protection:** the Helicopter unit is assumed to protect neighboring cells from fire propagation for 2 turns after deployment, instead of 1 turn.
- **Static civilian positions:** civilians are assumed to remain in their starting cell until the player explicitly triggers an evacuation. They do not move autonomously, seek shelter, or react to nearby fire between player actions.
- **BST insertion order:** the Binary Search Tree is assumed to be rebuilt from the current fire state at the start of each turn. It is assumed that the insertion order produces a sufficiently balanced tree for the problem scale ($n \leq 64$), making worst-case degeneration unlikely in practice.
- **JSON Bridge reliability:** communication between the C++ engine and the Python algorithm module via the JSON Bridge is assumed to be synchronous, lossless, and free of serialisation errors. No fault-tolerance or retry mechanism is considered in the design.
- **Single-threaded execution:** the entire game loop — engine update, algorithm execution, and UI rendering — is assumed to run on a single thread in a sequential, turn-based manner. Concurrency and race conditions are not addressed.
- **User competency:** it is assumed that the player understands the basic rules of the game before starting a session. No in-game tutorial, guided onboarding, or contextual help system is included in the proposed design.

5.2 Limitations

- **Design-only stage:** the most significant limitation of this work is that the game exists solely as a proposal. No implementation has been produced, which means that the correctness of the algorithms, the performance of the data structures, and the playability of the game mechanics have not been empirically validated. All claims about algorithmic behaviour are based on theoretical analysis alone.
- **No gameplay balancing:** the scoring values, unit arrival intervals, civilian point weights, and loss thresholds defined in the design have not been tested through actual play sessions. It is possible that certain difficulty configurations are trivially easy or impossible to win, and adjustments would likely be needed after playtesting.
- **Greedy suboptimality:** the Greedy dispatcher does not guarantee a globally optimal sequence of deployments. In scenarios where long-term planning would yield a significantly better outcome than greedy local decisions — particularly when managing Helicopter units — the algorithm may lead the player towards avoidable defeat without the player being aware of the suboptimal choice.
- **Static pathfinding:** the Backtracking evacuator computes routes based on the fire state at the moment of the decision, without accounting for future fire propagation. For slow civilians such as the elderly type, this may result in routes that become unsafe before the civilian completes them, a limitation that cannot be resolved without a time-aware pathfinding approach.
- **Fixed grid size:** the design is tightly coupled to an 8×8 grid. Extending the system to larger grids would require reassessing the BST in favour of a self-balancing structure, and would expose the exponential worst-case complexity of the Backtracking algorithm, potentially requiring replacement with BFS or A*.
- **Time constraints:** the scope of this project was bounded by academic deadlines, which limited the depth of exploration into alternative algorithmic approaches, more sophisticated fire propagation models, and extended difficulty configurations. Some design decisions reflect pragmatic choices made under time pressure rather than exhaustive optimisation.

Chapter 6

Methodology

This chapter describes the proposed design of the tactical fire simulation game, covering the system architecture, the data structures selected for the C++ engine, the algorithmic logic governing unit deployment and civilian evacuation, the communication bridge between the C++ and Python layers, and the game rules that define each difficulty level. All content in this chapter represents the intended design and has not yet been fully implemented.

6.1 System Architecture

The system is divided into three independent layers that communicate through defined interfaces:

- **C++ Engine:** manages the game state, including the 8×8 grid, fire propagation logic, the unit dispatch queue (linked list), and the risk-ranking structure (BST). At the end of each turn the engine serialises the current game state to a JSON object and passes it to the Python layer.
- **Python Algorithm Module:** receives the JSON game state, executes the Greedy dispatcher or the Backtracking evacuator depending on the player's action, and returns the resulting decision (target cell or evacuation route) to the engine as a JSON response.
- **Pygame Interface:** reads the game state produced by the engine after each turn and renders the grid, fire cells, civilian positions, unit queue, score HUD, and evacuation animations. All player input is captured here and forwarded to the engine.

The JSON Bridge is the sole communication channel between C++ and Python. Each message contains the full grid state, the list of active fires with their types and risk scores, the positions and types of all civilians, and the contents of the unit dispatch queue. This stateless interface keeps the three layers decoupled and allows each to be developed and tested independently.

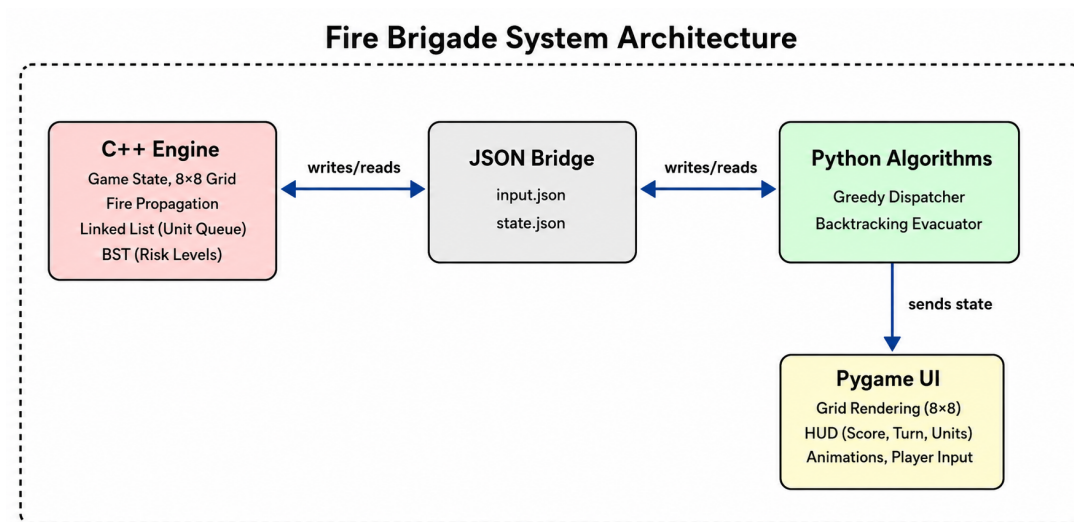


Figure 6.1: System Architecture: C++ Engine, JSON Bridge, Python Algorithms, and Pygame UI

6.2 Data Structure Design

6.2.1 Unit Dispatch Queue — Singly Linked List

The unit dispatch queue is implemented as a singly linked list in C++. Each node in the list represents one available firefighting unit and stores the following fields:

- `unit_type` (string): one of GRUPO, CAMION, or HELICOPTERO.
- `arrival_turn` (int): the turn on which the unit became available.
- `*next` (pointer): reference to the next node in the queue, or `nullptr` if this is the last unit.

New units are appended at the tail as they arrive according to the turn-based schedule defined by the difficulty level. The player dequeues from the head when selecting a unit to deploy. This FIFO structure preserves arrival order and supports $O(1)$ enqueue and dequeue operations.

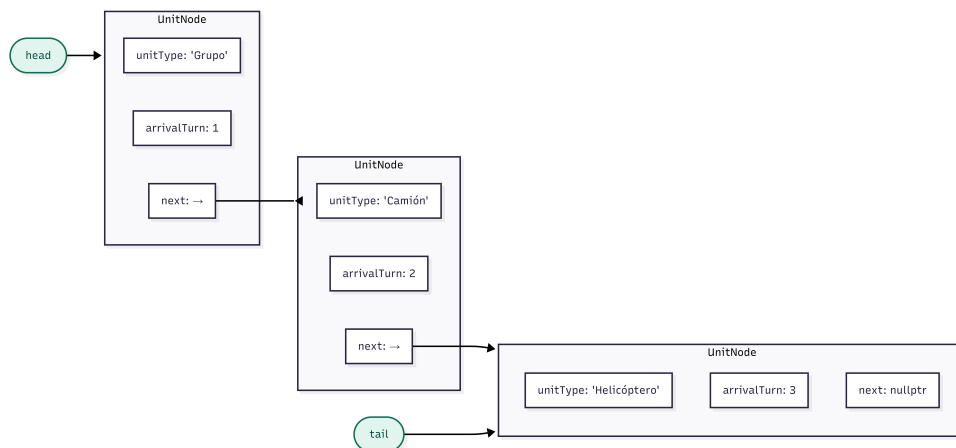


Figure 6.2: Linked List Structure for Unit Dispatch Queue

6.2.2 Risk-Ranking Structure — Binary Search Tree

The BST organises all currently burning cells ordered by their risk score. Each node stores:

- `risk_level` (float): the computed risk score, equal to the base intensity of the cell multiplied by 1.5 if the fire type is COPAS (canopy fire).
- `row`, `col` (int): grid coordinates of the burning cell.
- `fire_type` (string): NORMAL or COPAS.
- `*left`, `*right` (pointers): left and right child nodes.

The BST property (left subtree $<$ node $\leq$ right subtree by risk level) ensures that the maximum-risk cell is always the rightmost node, retrievable in $O(h)$ time. The tree is rebuilt after each turn to reflect newly ignited cells, extinguished cells, and updated risk scores caused by fire propagation.

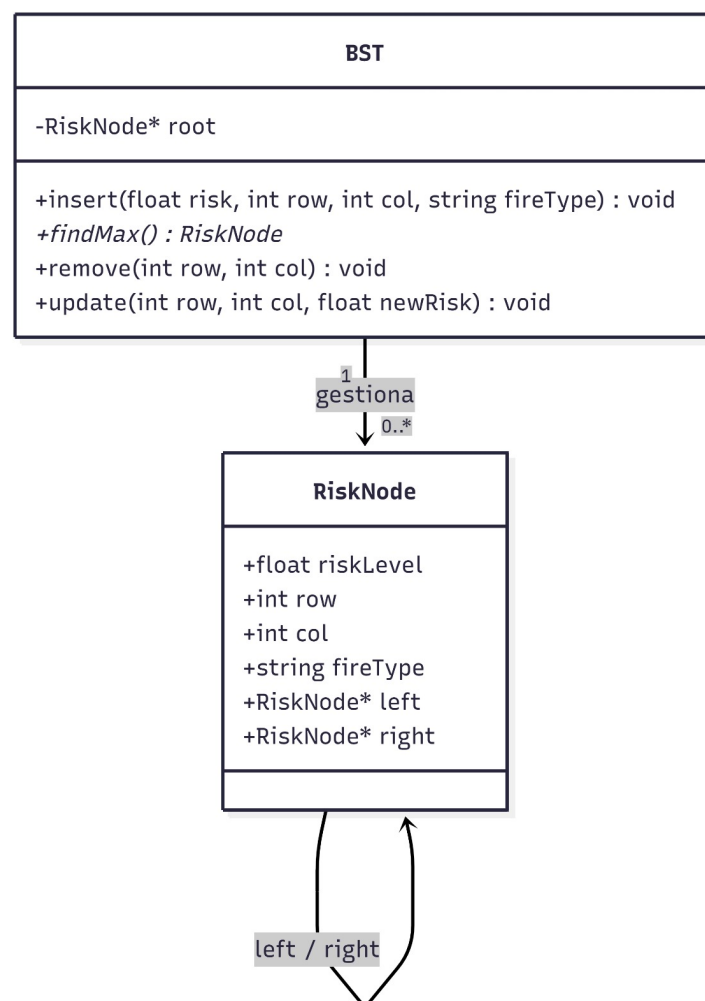


Figure 6.3: BST Node Structure and Class Methods

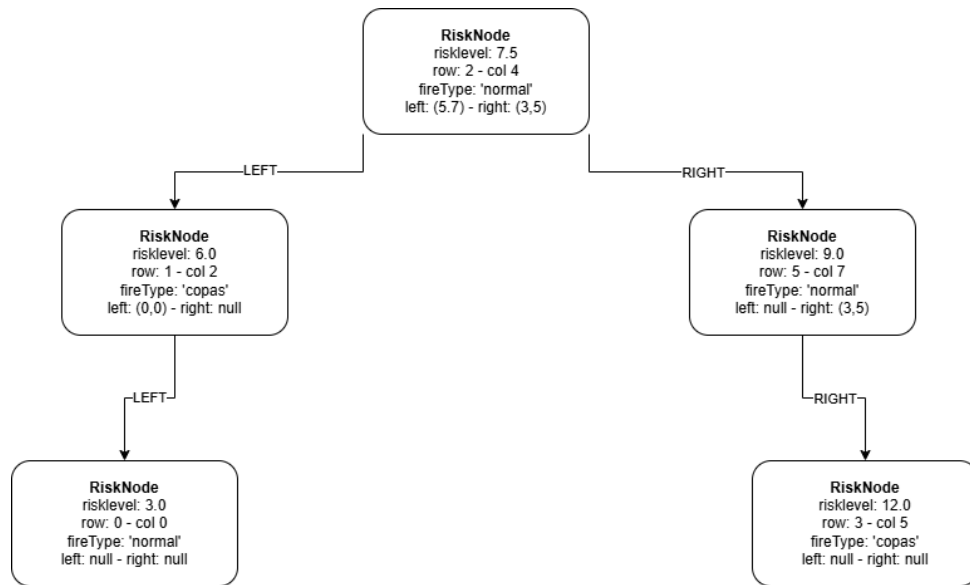


Figure 6.4: Example BST Nodes with Risk Values and Coordinates

6.3 Algorithm Design

6.3.1 Greedy Dispatcher

When the player chooses to deploy a unit, the **Greedy Dispatcher** algorithm is invoked. It takes the current BST and the selected unit type as input and returns the target cell coordinates.

1. Query the BST for the node with the maximum `risk_level` by traversing right-child pointers from the root.
2. If multiple cells share the maximum risk value, apply tie-breaking rules:
 - (a) Prefer any cell whose `fire_type` is COPAS, as canopy fires propagate more aggressively.
 - (b) If no canopy fire exists among the tied cells, select the cell with the smallest row index; if rows are equal, select the smallest column index.
3. Return the selected cell coordinates as the deployment destination.

The player retains strategic control over *which* unit to deploy and *when* to deploy it, but the destination is always determined by the Greedy algorithm. This design reflects the constraint that firefighting units in the field are dispatched to the most critical location rather than at the operator's discretion.

6.3.2 Backtracking Evacuator

When the player chooses to evacuate a civilian, the **Backtracking Evacuator** algorithm is invoked. It takes the civilian's starting cell, the obstacle matrix (cells currently on fire), and the civilian type as input, and returns the shortest safe path to any grid boundary cell, or reports failure if no such path exists.

The algorithm proceeds as follows:

1. Initialise an 8×8 boolean `visited` matrix to false, an empty `current_path` list, an empty `best_path` list, and `best_length` $\leftarrow$.
2. Call the recursive `explore` function from the civilian's starting position.

3. Inside `explore(position)`:
 - (a) Append `position` to `current_path`.
 - (b) *Base case*: if `position` is on the grid boundary (`row = 0`, `row = 7`, `column = 0`, or `column = 7`), compare `len(current_path)` with `best_length`; if shorter, update `best_path` and `best_length`.
 - (c) Mark `visited[position] ← true`.
 - (d) For each of the four cardinal neighbours (up, down, left, right): if the neighbour is within the grid, is not on fire, and has not been visited, recurse into `explore(neighbour)`.
 - (e) *Backtrack*: set `visited[position] ← false` and remove `position` from `current_path`.
4. After `explore` returns, if `best_path` is non-empty, return it with `success = true`; otherwise return an empty list with `success = false`.

The civilian type influences movement speed in the UI layer: an ANCIANO (elderly civilian) requires two turns to advance one cell along the returned path, while all other types advance one cell per turn. The algorithm itself is type-agnostic; speed is enforced by the engine when animating the route.

6.3.3 Complexity Analysis

Greedy Dispatcher

The Greedy dispatcher retrieves the highest-risk cell by traversing the BST from the root to the rightmost node. In a balanced BST, this operation has time complexity $O(\log n)$, where n is the number of burning cells (maximum 64). In the worst case (degenerate tree), complexity degrades to $O(n)$. However, given the random nature of fire propagation and risk updates, worst-case degeneration is unlikely in practice. Space complexity is $O(1)$ as no additional structures are allocated during the operation.

Backtracking Evacuator

The Backtracking evacuator performs a depth-first search on the 8×8 grid. In the worst case, it explores up to four directions per cell, leading to a theoretical time complexity of $O(4^n)$, where n is the number of cells (64).

The visited matrix prevents cycles and re-exploration of the same cell.

Early termination when a path reaches the border (though exploration continues to find shorter paths).

The grid size is fixed and small (64 cells), making exhaustive exploration feasible.

Space complexity is $O(n)$ for the visited matrix and recursion stack, with a maximum depth of 64.

Linked List Operations

The singly linked list used for the unit dispatch queue supports the following operations:

- **Enqueue (append at tail)**: $O(1)$ — the tail pointer allows direct access to the last node.
- **Dequeue (remove from head)**: $O(1)$ — the head pointer allows direct access to the first node.
- **Search**: $O(n)$ — not required for FIFO queue operations.
- **Space complexity**: $O(k)$, where k is the number of units in the queue (maximum 5-8 units in practice).

Binary Search Tree Operations

The BST managing risk levels supports the following operations:

- **Insert:** $O(\log n)$ average case, $O(n)$ worst case (degenerate tree).
- **FindMax (rightmost node):** $O(\log n)$ average case, $O(n)$ worst case.
- **Remove:** $O(\log n)$ average case, $O(n)$ worst case.
- **Space complexity:** $O(n)$, where n is the number of burning cells (maximum 64).

The choice of a standard BST over a self-balancing AVL tree was justified by the small input size ($n \leq 64$) and the random update pattern, making worst-case degeneration unlikely. The simpler implementation reduces code complexity and potential bugs.

6.4 Game Components

6.4.1 Unit Types

Three unit types are available, each with different effects and arrival rates:

Table 6.1: Unit Types and Characteristics

Unit	Effect	Arrival Rate (turns)
Crew	Extinguishes 1 cell + reduces neighbor risk by half	2–4
Truck	Extinguishes 1 cell + reduces neighbor risk by half	4–6
Helicopter	Extinguishes 1 cell + protects neighbors for 2 turns	6–8

6.4.2 Civilian Types

Four civilian types are present, each with different point values:

Table 6.2: Civilian Types and Values

Civilian Type	Point Value
Child	100
Adult	50
Elderly	75
Family Group	150

6.4.3 Fire Behavior

Two fire types with different spread patterns:

Table 6.3: Fire Types and Spread Patterns

Fire Type	Spread Directions	Risk Multiplier
Normal	4 directions (up, down, left, right)	1.0
Canopy	8 directions (including diagonals)	1.5

Propagation Rules:

- Fire propagates every **2 turns**, not every turn.
- Newly ignited cells inherit **80% of the source cell's intensity**.
- Protected cells (affected by a Helicopter) do not ignite during propagation.

6.5 Game Rules

This section defines the core gameplay rules that govern the simulation, including the scoring system, victory and defeat conditions, and the structure of each turn.

6.5.1 Scoring System

Points are awarded and deducted according to the following rules:

- Extinguishing a normal fire cell: $+10 \times \text{risk_level}$.
- Extinguishing a canopy fire cell: $+15 \times \text{risk_level}$.
- Evacuating a child: +100 points.
- Evacuating an adult: +50 points.
- Evacuating an elderly civilian: +75 points.
- Evacuating a family group: +150 points.
- Civilian burned: -100 points (civilian is removed from the map).
- All fires extinguished bonus: +200 points.
- All civilians evacuated bonus: +300 points.

6.5.2 Win Conditions

The player wins the game when both of the following conditions are satisfied simultaneously:

- All fires on the grid have been extinguished (no active burning cells remain).
- All civilians present at the start of the level have been successfully evacuated.

Upon victory, the final score is displayed, and the player is offered the option to proceed to the next difficulty level or return to the main menu.

6.5.3 Loss Conditions

The player loses the game if either of the following conditions occurs:

- **Civilian loss threshold exceeded:** more than half of the civilians present at the start of the level have been burned. For example:
 - With 4 initial civilians, the third civilian burned triggers defeat.
 - With 3 initial civilians, the second civilian burned triggers defeat.

- With 2 initial civilians, the second civilian burned triggers defeat.
- **Map saturation:** fire spreads to cover more than 80% of the grid (more than 51 out of 64 cells). This condition prevents indefinitely prolonged games where the player cannot win but also cannot lose.

Upon defeat, the final score is displayed, and the player may restart the level or return to the main menu.

6.5.4 Turn Structure

Each game turn follows this sequential order:

1. **Unit arrival:** any units scheduled to arrive on the current turn are appended to the tail of the linked list queue. Arrival intervals are defined by the difficulty level.
2. **Player actions:** the player may perform one or more actions while units remain available in the queue or civilians remain on the map:
 - *Deploy a unit:* the player selects a unit type from the queue; the Greedy algorithm determines the target cell (highest risk); the unit is dequeued. The fire at the target cell is extinguished, and the risk of neighboring cells is reduced by half.
 - *Evacuate a civilian:* the player selects a civilian on the grid; the Backtracking algorithm computes the shortest safe route to the border; the civilian begins moving along the route at their type-defined speed (elderly civilians require two turns per cell; others move one cell per turn).
3. **Fire propagation (every 2 turns):** the C++ engine expands each active fire cell to its neighbours according to its type:
 - *Normal fire:* propagates to the four cardinal directions (up, down, left, right).
 - *Canopy fire:* propagates to all eight neighbouring cells (including diagonals).

Newly ignited cells inherit **80% of the source cell's intensity**. Protected cells (affected by a Helicopter unit) do not ignite during this phase. Newly ignited cells are inserted into the BST with their computed risk scores.

4. **State update:** after player actions and propagation are completed, the engine updates the BST with any changes, checks win/loss conditions, serialises the new game state to JSON, and forwards it to the Pygame interface for rendering.

6.6 Difficulty Levels

Table 6.4 summarises the configuration of each difficulty level.

Table 6.4: Difficulty Level Configurations

Parameter	Easy	Medium	Hard
Normal fires	2	2	2
Canopy fires	0	1	3
Civilians	2	3	4
Initial units	Crew, Truck, Heli	Crew, Truck, Heli	Crew, Truck
Crew arrival (turns)	2	3	4
Truck arrival (turns)	4	5	6
Helicopter arrival (turns)	6	7	8
Fire propagation	Normal only	Mixed	Aggressive

On the Easy level, only normal fire (4-directional propagation) is present. On Medium, one canopy fire cell is introduced, adding 8-directional propagation and a $\times 1.5$ risk weight. On Hard, three canopy fires are active from the start, no helicopter is provided initially, and all unit arrival intervals are at their longest, placing maximum pressure on the player's resource management.

6.7 JSON Communication Protocol

6.7.1 input.json (Python $\rightarrow$ C++)

This file is generated by Python on each turn and contains only the action selected by the player and the necessary parameters.

```
{
  "turn": 12,
  "difficulty": "HARD",
  "action": "DEPLOY",
  "parameters": {"unit_id": 2}
}
```

6.7.2 state.json (C++ $\rightarrow$ Python)

This file is generated by the engine after updating the game state and provides all necessary information for Python to execute the algorithms.

```
{
  "grid_size": [8, 8],
  "fires": [
    { "position": [2, 3], "type": "NORMAL", "intensity": 50 },
    { "position": [5, 4], "type": "COPAS", "intensity": 90 }
  ],
  "units": [
    { "id": 2, "type": "CAMION", "status": "READY" }
  ],
  "civilians": [
    { "id": 205, "type": "ANCIANO", "position": [4, 5], "status": "ALIVE" }
  ],
  "risk_zones": [
```

```

    { "position": [2, 3], "risk": 50 },
    { "position": [5, 4], "risk": 135 }
  ],
  "game_stats": {
    "score": 1250,
    "turn": 12,
    "civilians_alive": 3,
    "civilians_dead": 1,
    "defeat": false
  }
}

```

6.8 User Interface Wireframe

The Pygame interface presents an 8×8 grid with color-coded cells representing forest (green), normal fire (red), canopy fire (dark red), and civilians (blue). The HUD displays current turn, score, available units with individual deploy buttons, and action buttons for evacuation and ending the turn.



Figure 6.5: User Interface Wireframe: 8×8 Grid and HUD Panel

Chapter 7

Implementation & Results

7.1 Implementation Status

As of the submission of this report, the project is in the **design and planning phase**. The complete system architecture, data structures, algorithms, game rules, and communication protocols have been fully specified and documented as presented in the previous chapters. However, the final implementation in C++, Python, and Pygame has not yet been completed.

The following components have been designed and are ready for implementation:

- **C++ Engine:** Specification of the 8×8 grid, fire propagation logic, unit dispatch queue (singly linked list), and risk-ranking structure (Binary Search Tree). The JSON serialization and deserialization logic for communication with Python is also defined.
- **Python Algorithm Module:** Pseudocode and logic for the Greedy dispatcher (including tie-breaking rules) and the Backtracking evacuator (including recursive exploration) have been fully specified.
- **Pygame Interface:** The wireframe for the 8×8 grid, HUD elements, and interaction flow has been designed. The visual representation of fires (normal and canopy), civilians (four types), and unit controls has been defined.
- **JSON Communication Protocol:** The schemas for `input.json` and `state.json` have been defined, including all required fields, data types, and example values.
- **Game Rules and Difficulty Levels:** The scoring system, win/loss conditions, turn structure, and three difficulty levels (Easy, Medium, Hard) have been fully specified.

7.2 Planned Implementation

The following steps constitute the planned implementation roadmap:

1. **Phase 1 – Core Engine (C++):** Implement the 8×8 grid, fire propagation logic, linked list for unit queue, BST for risk ranking, and JSON serialization.
2. **Phase 2 – Algorithms (Python):** Implement the Greedy dispatcher and Backtracking evacuator, including tie-breaking logic and recursive pathfinding.
3. **Phase 3 – User Interface (Pygame):** Build the graphical interface with grid rendering, HUD, unit controls, and evacuation animations.

4. **Phase 4 – Integration:** Connect C++ and Python via JSON files, ensuring correct state propagation and action execution.
5. **Phase 5 – Testing and Balancing:** Conduct unit tests, integration tests, and gameplay balancing across difficulty levels.

7.3 Current Implementation Screenshots

The following figures show the current state of the implemented Pygame interface, demonstrating the main menu, the 8×8 game grid, and the win/loss conditions.

7.3.1 Main Menu

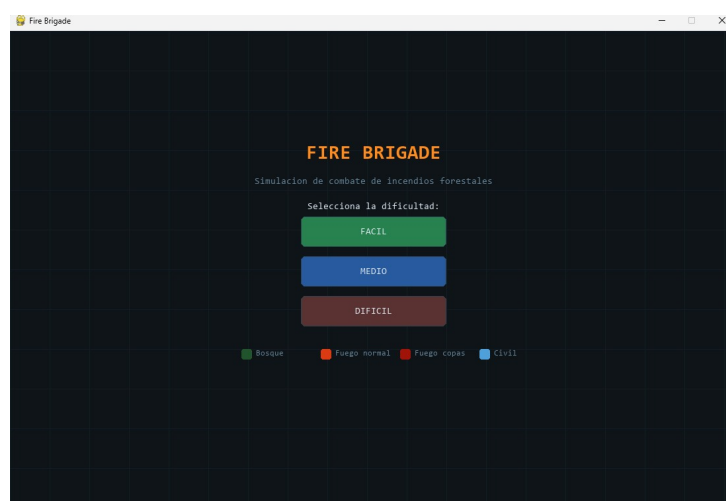


Figure 7.1: Main Menu Screen

7.3.2 Game Grid

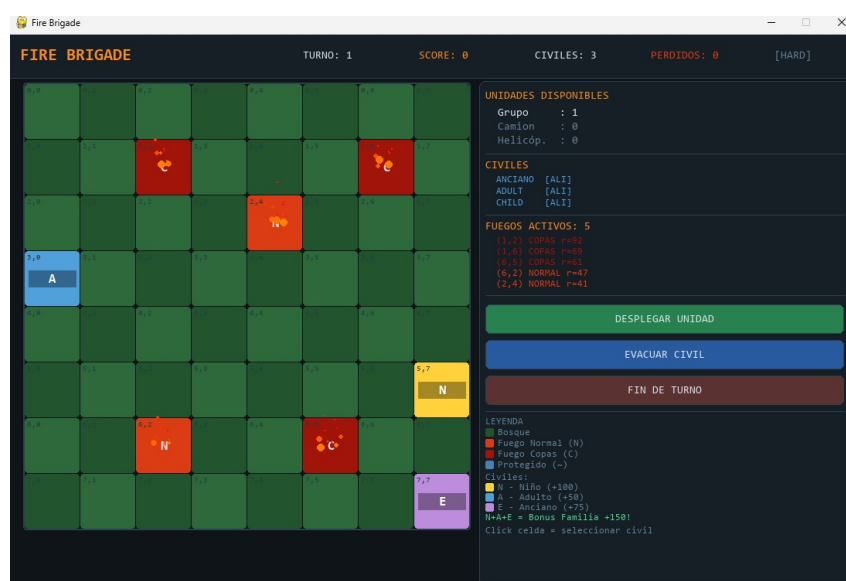


Figure 7.2: 8×8 Game Grid with Active Fires and Civilians

7.3.3 Victory Screen

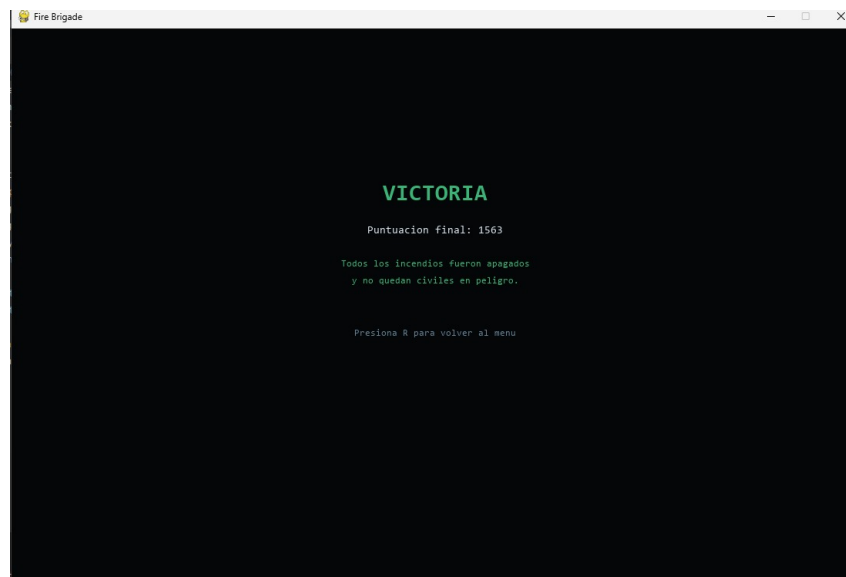


Figure 7.3: Victory Screen – All Fires Extinguished and Civilians Evacuated

7.3.4 Defeat Screen

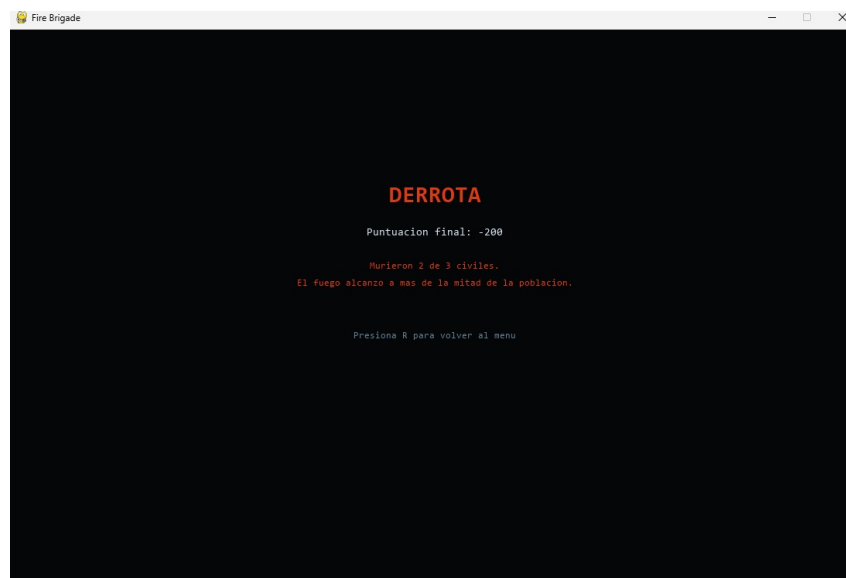


Figure 7.4: Defeat Screen – Loss Condition Triggered

7.4 Expected Results

Upon completion of the implementation, the following results are expected:

- A fully playable turn-based simulation game where the player commands firefighting and evacuation operations.
- A Greedy dispatcher that consistently sends units to the highest-risk cell, with documented tie-breaking behavior.

- A Backtracking evacuator that reliably finds shortest safe routes to the grid boundary or correctly reports failure.
- A BST that maintains burning cells ordered by risk, enabling $O(\log n)$ retrieval of the maximum-risk cell.
- A linked list that manages unit arrivals and deployments with FIFO ordering.
- Functional JSON communication between C++ and Python, enabling independent development and testing of each layer.
- Three difficulty levels that provide increasing challenge and require strategic resource management.
- A scoring system that rewards efficient fire suppression and civilian evacuation while penalizing civilian casualties.

7.5 Current Results

As this report represents a design deliverable submitted during the development phase, no empirical results or performance measurements are available at this stage. The work presented here focuses on the complete specification of the system, including:

- Full pseudocode for both algorithms (Greedy and Backtracking)
- Complete data structure specifications with diagrams
- Detailed JSON schemas with examples
- Comprehensive game rules and difficulty configurations
- Wireframe design for the Pygame interface
- Current implementation screenshots of the main menu, game grid, and win/loss conditions

Once implementation is complete, the following metrics will be evaluated and reported:

- **Algorithm correctness:** Percentage of cases where Greedy selects the correct highest-risk cell and Backtracking finds a valid route when one exists.
- **Performance:** Execution time for BST operations (insert, findMax, remove) and Backtracking pathfinding across different fire configurations.
- **Gameplay balance:** Win rates across difficulty levels and player feedback on strategic depth.
- **JSON communication:** Latency and reliability of state serialization and deserialization.

These results will be presented in a future version of this report upon completion of the implementation phase.

Chapter 8

Conclusion and Reflections

8.1 Conclusion

This report has presented the complete design of Fire Brigade, a turn-based tactical simulation game that implements Greedy and Backtracking algorithms within a modular C++/Python/Pygame architecture. The following components have been fully specified:

- A **singly linked list** for managing the unit dispatch queue with FIFO ordering and $O(1)$ enqueue/dequeue operations.
- A **Binary Search Tree (BST)** for maintaining burning cells ordered by risk level, enabling $O(\log n)$ retrieval of the highest-risk cell for the Greedy dispatcher.
- A **Greedy dispatcher** that automatically sends units to the highest-risk cell, with deterministic tie-breaking rules (canopy fires prioritized, then smallest coordinates).
- A **Backtracking evacuator** that finds shortest safe evacuation routes to the grid boundary using depth-first search with pruning.
- **Three difficulty levels** (Easy, Medium, Hard) with varying initial conditions for fires, civilians, units, and arrival rates.
- **Three gameplay variants**: scarce resources (unit arrival timers), civilian priorities (different point values and movement speeds), and dynamic fire behavior (normal and canopy propagation).
- A **JSON communication protocol** (input.json and state.json) defining the interface between C++ and Python.
- A **Pygame wireframe** with 8×8 grid, HUD, and interactive controls.

The design phase is complete, and the project is ready to proceed to implementation.

8.2 Reflections

8.2.1 Design Decisions

Several key design decisions were made during the development of this project:

- **BST over AVL:** A standard BST was chosen over a self-balancing AVL tree because the grid contains at most 64 cells. The practical performance difference is negligible, and the simpler BST implementation reduces code complexity and potential bugs. Given that risk values change randomly due to fire propagation, worst-case degeneration is unlikely.
- **Greedy as automatic dispatcher:** The Greedy algorithm is mandatory and always determines the destination of deployed units. This decision was driven by the requirement to demonstrate algorithmic decision-making as a core mechanic. The player retains strategic control over *which* unit to deploy and *when* to deploy it, but the algorithm decides *where*.
- **Backtracking for evacuation:** Backtracking was chosen because it guarantees finding the shortest path when one exists. For an 8×8 grid, exhaustive search is computationally feasible, and the algorithm's behavior is easy to explain and observe.
- **JSON file I/O for communication:** This stateless approach keeps C++ and Python completely decoupled, allowing independent development and testing of each layer. No external libraries or complex bindings are required.
- **Game balance:** after evaluating initial design considerations, fire propagation was adjusted to occur every 2 turns instead of every turn, and newly ignited cells inherit 80% of the source cell's intensity. Additionally, Crew and Truck units now reduce the risk of neighboring cells by half, and the Helicopter's protection duration was extended from 1 to 2 turns. These changes make the game more forgiving and strategically deeper, giving players more time to respond while maintaining challenge.

8.2.2 Challenges Encountered

The main challenges during the design phase included:

- **Maintaining consistency across layers:** Ensuring that the JSON schemas for input.json and state.json accurately represent all game state information required by both C++ and Python.
- **Tie-breaking in Greedy:** Defining deterministic rules for cases where multiple cells share the same risk value, ensuring predictable algorithm behavior.
- **Balancing difficulty levels:** Configuring initial fires, civilians, units, and arrival rates to provide meaningful progression from Easy to Hard without making any level impossible or trivial.
- **Civilian movement speed:** Incorporating the elderly civilian's slower movement (two turns per cell) without complicating the Backtracking algorithm itself, handled instead by the engine during animation.

8.2.3 Lessons Learned

This project provided valuable insights into:

- **Integration of classical algorithms into interactive systems:** Making abstract concepts tangible and observable through gameplay.
- **Separation of concerns:** How a modular architecture (C++ engine, Python algorithms, Pygame UI) with a simple communication protocol (JSON) enables parallel development.

- **Data structure selection:** How problem scale influences the choice between simple structures (BST) and self-balancing variants (AVL).
- **Deterministic algorithm design:** The importance of explicit tie-breaking rules for reproducibility and predictability.

8.3 Future Work

Upon completion of the implementation phase, the following tasks remain:

- **Full implementation:** Coding the C++ engine, Python algorithms, and Pygame interface according to the specifications.
- **Unit and integration testing:** Validating the correctness of all data structures and algorithms.
- **Gameplay balancing:** Adjusting scoring values, unit arrival intervals, and loss thresholds based on playtesting feedback.
- **Performance benchmarking:** Measuring actual runtime complexity against theoretical expectations.
- **Potential extensions:** Save/load functionality, tutorial mode, or additional map configurations.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] J. Kleinberg and E. Tardos, *Algorithm Design*. Boston, MA: Addison-Wesley, 2005.
- [3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.
- [4] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. London: Springer, 2008.
- [5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.
- [6] B. Stroustrup, *The C++ Programming Language*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [7] M. Lutz, *Learning Python*, 5th ed. Sebastopol, CA: O'Reilly Media, 2013.
- [8] Pygame Community, "Pygame — Python game development," <https://www.pygame.org>, 2024, accessed: 2026.
- [9] Ecma International, "ECMA-404: The JSON data interchange standard," <https://www.ecma-international.org/publications-and-standards/standards/ecma-404>, 2017, accessed: 2026.